Phala Network 배포 및 운영 가이드

1. 배포 개요

Phala Network의 배포는 **다중 컴포넌트 시스템**으로, 블록체인 노드, pRuntime, Pherry 릴레이어, 그리고 모니터링 시스템을 포함한 전체 스택의 조율된 배포가 필요합니다.

2. 배포 아키텍처

2.1 프로덕션 환경 구조

```
Production Deployment Architecture
├── Load Balancer (nginx/HAProxy)
| ├── RPC Endpoints (19933, 19944)
| └── pRuntime Endpoints (18000)
|
├── Blockchain Layer
| ├── Validator Nodes (Multiple)
| ├── Full Nodes (Archive/Pruned)
| └── Boot Nodes
|
├── TEE Computing Layer
| ├── pRuntime Instances (SGX)
```

```
| ├── Worker Clusters
| └── SideVM Instances
|
├── Bridge Layer
| ├── Pherry Relayers (Multiple)
| ├── PRB Instances
| └── Backup Relayers
|
├── Monitoring & Logging
| ├── Prometheus + Grafana
| ├── ELK Stack
| └── Alert Manager
|
└── Infrastructure
├── Container Orchestration (K8s/Docker)
├── Service Discovery
├── Secret Management
└── Backup Systems
```

2.2 네트워크 토폴로지

```
Internet
↓
```

```
Load Balancer (SSL Termination)
↓
┌──────────────────────────────────────────┐
│ DMZ Network │
│ ┌──────────────┐ ┌──────────────┐ │
│ │ RPC Gateway │ │ API Gateway │ │
│ └──────────────┘ └──────────────┘ │
└──────────────────────────────────────────┘
↓
┌──────────────────────────────────────────┐
│ Internal Network │
│ ┌──────────┐ ┌──────────┐ ┌──────────┐ │
│ │Blockchain│ │pRuntime │ │Pherry │ │
│ │ Nodes │ │Instances │ │Relayers │ │
│ └──────────┘ └──────────┘ └──────────┘ │
└──────────────────────────────────────────┘
↓
┌──────────────────────────────────────────┐
│ Storage Network │
│ ┌──────────┐ ┌──────────┐ ┌──────────┐ │
│ │Chain DB │ │TEE State │ │Logs & │ │
│ │(RocksDB) │ │Storage │ │Metrics │ │
│ └──────────┘ └──────────┘ └──────────┘ │
```

└──┘

3. Docker 기반 배포

3.1 Dockerfile 구성

```
# Phala Node Dockerfile
FROM ubuntu:22.04 as builder

# 의존성 설치
RUN apt-get update && apt-get install -y \
build-essential \
pkg-config \
libssl-dev \
protobuf-compiler \
curl \
git

# Rust 설치
RUN curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
ENV PATH="/root/.cargo/bin:${PATH}"

# 소스 코드 복사 및 빌드
```

```
WORKDIR /phala

COPY . .

RUN git submodule update --init --recursive

RUN cargo build --release --bin phala-node


# 런타임 이미지

FROM ubuntu:22.04


RUN apt-get update && apt-get install -y \

ca-certificates \

curl \

&& rm -rf /var/lib/apt/lists/*


# 바이너리 복사

COPY --from=builder /phala/target/release/phala-node /usr/local/bin/


# 사용자 생성

RUN useradd -m -u 1000 phala

USER phala


# 데이터 디렉토리

VOLUME ["/data"]

WORKDIR /data
```

```
# 포트 노출
EXPOSE 30333 19933 19944

# 헬스체크
HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \
CMD curl -f http://localhost:19933/health || exit 1

# 엔트리포인트
ENTRYPOINT ["/usr/local/bin/phala-node"]
```

3.2 pRuntime Dockerfile

```
# pRuntime Dockerfile (SGX 지원)
FROM ubuntu:22.04 as sgx-base

# SGX 의존성 설치
RUN apt-get update && apt-get install -y \
wget \
gnupg \
software-properties-common

# Intel SGX APT 저장소 추가
```

```
RUN wget -qO - https://download.01.org/intel-sgx/sgx_repo/ubuntu/intel-sgx-deb.k

RUN add-apt-repository "deb https://download.01.org/intel-sgx/sgx_repo/ubuntu $(


# SGX 드라이버 및 SDK 설치

RUN apt-get update && apt-get install -y \

sgx-aesm-service \

libsgx-aesm-launch-plugin \

libsgx-aesm-pce-plugin \

libsgx-aesm-quote-ex-plugin \

libsgx-urts \

libsgx-dcap-ql


FROM sgx-base as builder


# 빌드 의존성

RUN apt-get install -y \

build-essential \

pkg-config \

libssl-dev \

protobuf-compiler \

curl \

git \

cmake
```

```
# Rust 설치
RUN curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
ENV PATH="/root/.cargo/bin:${PATH}"

# pRuntime 빌드
WORKDIR /phala
COPY . .
WORKDIR /phala/standalone/pruntime
RUN make

# 런타임 이미지
FROM sgx-base

# pRuntime 바이너리 복사
COPY --from=builder /phala/standalone/pruntime/bin/pruntime /usr/local/bin/
COPY --from=builder /phala/standalone/pruntime/bin/*.so /usr/local/lib/

# 환경 설정
ENV LD_LIBRARY_PATH="/usr/local/lib:${LD_LIBRARY_PATH}"
ENV RUST_LOG="info"

# 사용자 생성
```

```
RUN useradd -m -u 1000 pruntime

USER pruntime


# 데이터 디렉토리

VOLUME ["/data"]

WORKDIR /data


# 포트 노출

EXPOSE 18000


# 헬스체크

HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \

CMD curl -f http://localhost:18000/get_info || exit 1


# 엔트리포인트

ENTRYPOINT ["/usr/local/bin/pruntime"]
```

3.3 Docker Compose 설정

```
# docker-compose.yml

version: '3.8'


services:
```

```yaml
phala-node:
build:
context: .
dockerfile: Dockerfile.node
container_name: phala-node
ports:
- "30333:30333"  # P2P
- "19933:19933"  # RPC
- "19944:19944"  # WebSocket
volumes:
- phala-data:/data
- ./chain-spec.json:/chain-spec.json:ro
environment:
- RUST_LOG=info
command: >
--chain /chain-spec.json
--base-path /data
--rpc-external
--ws-external
--rpc-cors all
--prometheus-external
--prometheus-port 9615
restart: unless-stopped
```

```yaml
networks:
- phala-network


pruntime:
build:
context: .
dockerfile: Dockerfile.pruntime
container_name: pruntime
ports:
- "18000:18000"
volumes:
- pruntime-data:/data
- /dev/sgx_enclave:/dev/sgx_enclave
- /dev/sgx_provision:/dev/sgx_provision
environment:
- RUST_LOG=info
- ROCKET_ADDRESS=0.0.0.0
- ROCKET_PORT=18000
privileged: true  # SGX 접근을 위해 필요
restart: unless-stopped
networks:
- phala-network
depends_on:
```

```
- phala-node


pherry:
build:
context: .
dockerfile: Dockerfile.pherry
container_name: pherry
environment:
- RUST_LOG=info
- SUBSTRATE_WS_ENDPOINT=ws://phala-node:19944
- PRUNTIME_ENDPOINT=http://pruntime:18000
- MNEMONIC=${PHERRY_MNEMONIC}
restart: unless-stopped
networks:
- phala-network
depends_on:
- phala-node
- pruntime


prometheus:
image: prom/prometheus:latest
container_name: prometheus
ports:
```

```yaml
- "9090:9090"
volumes:
- ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
- prometheus-data:/prometheus
command:
- '--config.file=/etc/prometheus/prometheus.yml'
- '--storage.tsdb.path=/prometheus'
- '--web.console.libraries=/etc/prometheus/console_libraries'
- '--web.console.templates=/etc/prometheus/consoles'
restart: unless-stopped
networks:
- phala-network


grafana:
image: grafana/grafana:latest
container_name: grafana
ports:
- "3000:3000"
volumes:
- grafana-data:/var/lib/grafana
- ./monitoring/grafana/dashboards:/etc/grafana/provisioning/dashboards:ro
- ./monitoring/grafana/datasources:/etc/grafana/provisioning/datasources:ro
environment:
```

```yaml
    - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD}
    restart: unless-stopped
    networks:
    - phala-network

volumes:
  phala-data:
  pruntime-data:
  prometheus-data:
  grafana-data:

networks:
  phala-network:
    driver: bridge
```

4. Kubernetes 배포

4.1 Namespace 및 ConfigMap

```yaml
# k8s/namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
```

```yaml
name: phala-network
labels:
name: phala-network


---
# k8s/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
name: phala-config
namespace: phala-network
data:
chain-spec.json: |
{
"name": "Phala Network",
"id": "phala",
"chainType": "Live",
"bootNodes": [
"/dns/bootnode1.phala.network/tcp/30333/p2p/12D3KooW...",
"/dns/bootnode2.phala.network/tcp/30333/p2p/12D3KooW..."
],
"properties": {
"tokenDecimals": 12,
```

```
"tokenSymbol": "PHA"
}
}
prometheus.yml: |
global:
scrape_interval: 15s
scrape_configs:
- job_name: 'phala-node'
static_configs:
- targets: ['phala-node:9615']
- job_name: 'pruntime'
static_configs:
- targets: ['pruntime:18000']
metrics_path: '/metrics'
```

4.2 Phala Node Deployment

```
# k8s/phala-node-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
name: phala-node
namespace: phala-network
```

```yaml
labels:
app: phala-node
spec:
replicas: 3
selector:
matchLabels:
app: phala-node
template:
metadata:
labels:
app: phala-node
spec:
containers:
- name: phala-node
image: phala/node:latest
ports:
- containerPort: 30333
name: p2p
- containerPort: 19933
name: rpc
- containerPort: 19944
name: ws
- containerPort: 9615
```

```yaml
name: metrics
env:
- name: RUST_LOG
value: "info"
volumeMounts:
- name: chain-data
mountPath: /data
- name: chain-spec
mountPath: /chain-spec.json
subPath: chain-spec.json
command:
- /usr/local/bin/phala-node
args:
- --chain
- /chain-spec.json
- --base-path
- /data
- --rpc-external
- --ws-external
- --rpc-cors
- all
- --prometheus-external
- --prometheus-port
```

```
- "9615"
resources:
requests:
memory: "2Gi"
cpu: "1000m"
limits:
memory: "4Gi"
cpu: "2000m"
livenessProbe:
httpGet:
path: /health
port: 19933
initialDelaySeconds: 60
periodSeconds: 30
readinessProbe:
httpGet:
path: /health
port: 19933
initialDelaySeconds: 30
periodSeconds: 10
volumes:
- name: chain-data
persistentVolumeClaim:
```

```yaml
claimName: phala-node-pvc

- name: chain-spec

configMap:

name: phala-config



---

apiVersion: v1

kind: Service

metadata:

name: phala-node

namespace: phala-network

spec:

selector:

app: phala-node

ports:

- name: p2p

port: 30333

targetPort: 30333

protocol: TCP

- name: rpc

port: 19933

targetPort: 19933

- name: ws
```

```yaml
port: 19944
targetPort: 19944
- name: metrics
port: 9615
targetPort: 9615
type: ClusterIP


---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: phala-node-pvc
namespace: phala-network
spec:
accessModes:
- ReadWriteOnce
resources:
requests:
storage: 100Gi
storageClassName: fast-ssd
```

4.3 pRuntime StatefulSet

```yaml
# k8s/pruntime-statefulset.yaml

apiVersion: apps/v1

kind: StatefulSet

metadata:

name: pruntime

namespace: phala-network

spec:

serviceName: pruntime

replicas: 2

selector:

matchLabels:

app: pruntime

template:

metadata:

labels:

app: pruntime

spec:

nodeSelector:

sgx.intel.com/sgx: "true"  # SGX 지원 노드에만 배포

containers:

- name: pruntime

image: phala/pruntime:latest
```

```yaml
ports:
- containerPort: 18000
name: http
env:
- name: RUST_LOG
value: "info"
- name: ROCKET_ADDRESS
value: "0.0.0.0"
- name: ROCKET_PORT
value: "18000"
volumeMounts:
- name: pruntime-data
mountPath: /data
- name: sgx-devices
mountPath: /dev/sgx_enclave
- name: sgx-provision
mountPath: /dev/sgx_provision
securityContext:
privileged: true  # SGX 접근을 위해 필요
resources:
requests:
memory: "4Gi"
cpu: "2000m"
```

```
limits:
memory: "8Gi"
cpu: "4000m"
livenessProbe:
httpGet:
path: /get_info
port: 18000
initialDelaySeconds: 60
periodSeconds: 30
readinessProbe:
httpGet:
path: /get_info
port: 18000
initialDelaySeconds: 30
periodSeconds: 10
volumes:
- name: sgx-devices
hostPath:
path: /dev/sgx_enclave
- name: sgx-provision
hostPath:
path: /dev/sgx_provision
volumeClaimTemplates:
```

```yaml
- metadata:
name: pruntime-data
spec:
accessModes: ["ReadWriteOnce"]
resources:
requests:
storage: 50Gi
storageClassName: fast-ssd


---
apiVersion: v1
kind: Service
metadata:
name: pruntime
namespace: phala-network
spec:
selector:
app: pruntime
ports:
- name: http
port: 18000
targetPort: 18000
```

```
clusterIP: None  # Headless service for StatefulSet
```

5. 모니터링 및 로깅

5.1 Prometheus 설정

```
# monitoring/prometheus.yml
global:
scrape_interval: 15s
evaluation_interval: 15s

rule_files:
- "phala_rules.yml"

alerting:
alertmanagers:
- static_configs:
- targets:
- alertmanager:9093

scrape_configs:
- job_name: 'phala-node'
static_configs:
```

```
- targets: ['phala-node:9615']
scrape_interval: 10s
metrics_path: '/metrics'

- job_name: 'pruntime'
static_configs:
- targets: ['pruntime:18000']
scrape_interval: 10s
metrics_path: '/metrics'

- job_name: 'pherry'
static_configs:
- targets: ['pherry:8080']
scrape_interval: 30s

- job_name: 'kubernetes-pods'
kubernetes_sd_configs:
- role: pod
namespaces:
names:
- phala-network
relabel_configs:
- source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
```

```yaml
action: keep

regex: true
```

5.2 Alert Rules

```yaml
# monitoring/phala_rules.yml

groups:

- name: phala.rules

rules:

- alert: PhalaNodeDown

expr: up{job="phala-node"} == 0

for: 5m

labels:

severity: critical

annotations:

summary: "Phala node is down"

description: "Phala node {{ $labels.instance }} has been down for more than 5 mi



- alert: PruntimeDown

expr: up{job="pruntime"} == 0

for: 2m

labels:

severity: critical
```

```
annotations:
summary: "pRuntime is down"
description: "pRuntime {{ $labels.instance }} has been down for more than 2 minu


- alert: BlockProductionSlow
expr: increase(substrate_block_height[5m]) < 10
for: 5m
labels:
severity: warning
annotations:
summary: "Block production is slow"
description: "Block production rate is below expected threshold."


- alert: HighMemoryUsage
expr: (node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes) / node_memor
for: 5m
labels:
severity: warning
annotations:
summary: "High memory usage"
description: "Memory usage is above 90% on {{ $labels.instance }}."
```

5.3 Grafana 대시보드

```
{
"dashboard": {
"title": "Phala Network Overview",
"panels": [
{
"title": "Block Height",
"type": "stat",
"targets": [
{
"expr": "substrate_block_height",
"legendFormat": "Block Height"
}
]
},
{
"title": "Active Workers",
"type": "stat",
"targets": [
{
"expr": "phala_active_workers",
"legendFormat": "Active Workers"
}
```

```
]
},
{
"title": "Transaction Pool",
"type": "graph",
"targets": [
{
"expr": "substrate_txpool_ready_transactions",
"legendFormat": "Ready Transactions"
},
{
"expr": "substrate_txpool_future_transactions",
"legendFormat": "Future Transactions"
}
]
},
{
"title": "pRuntime Status",
"type": "table",
"targets": [
{
"expr": "pruntime_info",
"legendFormat": "{{ instance }}"
```

```
}
]
}
]
}
}
```

6. 백업 및 복구

6.1 데이터 백업 전략

```bash
#!/bin/bash
# scripts/backup.sh

set -e

BACKUP_DIR="/backup/phala-$(date +%Y%m%d-%H%M%S)"
RETENTION_DAYS=30

echo "🔄 Starting Phala Network backup..."

# 1. 블록체인 데이터 백업
backup_blockchain_data() {
```

```
echo "📦 Backing up blockchain data..."
# 노드 일시 정지 (선택적)
# kubectl scale deployment phala-node --replicas=0 -n phala-network
# 데이터 압축 백업
tar -czf "${BACKUP_DIR}/blockchain-data.tar.gz" \
-C /var/lib/phala/node \
chains/
# 노드 재시작
# kubectl scale deployment phala-node --replicas=3 -n phala-network
}


# 2. pRuntime 상태 백업
backup_pruntime_state() {
echo "🔒 Backing up pRuntime state..."
# 각 pRuntime 인스턴스의 상태 백업
for instance in $(kubectl get pods -l app=pruntime -n phala-network -o
pod_name=$(basename $instance)
# 상태 파일 백업
kubectl exec $instance -n phala-network -- \
tar -czf - /data/protected_files /data/storage_files | \
cat > "${BACKUP_DIR}/pruntime-${pod_name}.tar.gz"
done
}
```

```
# 3. 설정 파일 백업
backup_configs() {
echo "⚙️ Backing up configurations..."
# Kubernetes 매니페스트
kubectl get all,configmap,secret,pvc -n phala-network -o yaml > \
"${BACKUP_DIR}/k8s-manifests.yaml"
# 체인 스펙
cp /etc/phala/chain-spec.json "${BACKUP_DIR}/"
# 모니터링 설정
cp -r /etc/prometheus "${BACKUP_DIR}/prometheus-config"
cp -r /etc/grafana "${BACKUP_DIR}/grafana-config"
}

# 4. 메타데이터 생성
create_metadata() {
echo "📋 Creating backup metadata..."
cat > "${BACKUP_DIR}/metadata.json" << EOF
{
"timestamp": "$(date -u +%Y-%m-%dT%H:%M:%SZ)",
"version": "$(kubectl get deployment phala-node -n phala-network -o jsonpath='{.
"block_height": $(curl -s http://phala-node:19933 -H "Content-Type: application/
"backup_size": "$(du -sh ${BACKUP_DIR} | cut -f1)"
```

```
}
EOF
}

# 5. 오래된 백업 정리
cleanup_old_backups() {
echo "🧹 Cleaning up old backups..."
find /backup -name "phala-*" -type d -mtime +${RETENTION_DAYS} -exec rm
}

# 메인 실행
main() {
mkdir -p "${BACKUP_DIR}"
backup_blockchain_data
backup_pruntime_state
backup_configs
create_metadata
cleanup_old_backups
echo "✅ Backup completed: ${BACKUP_DIR}"
echo "📊 Backup size: $(du -sh ${BACKUP_DIR} | cut -f1)"
}
```

```
main "$@"
```

6.2 재해 복구 절차

```
#!/bin/bash
# scripts/disaster-recovery.sh

set -e

BACKUP_PATH="$1"
RECOVERY_MODE="${2:-full}"  # full, partial, config-only

if [ -z "$BACKUP_PATH" ]; then
echo "Usage: $0 <backup_path> [recovery_mode]"
exit 1
fi

echo "🚨 Starting disaster recovery from: $BACKUP_PATH"
echo "🔧 Recovery mode: $RECOVERY_MODE"

# 1. 환경 검증
validate_environment() {
echo "✅ Validating recovery environment..."
```

```
# Kubernetes 클러스터 접근 확인
if ! kubectl cluster-info > /dev/null 2>&1; then
echo "❌ Cannot access Kubernetes cluster"
exit 1
fi
# 백업 파일 존재 확인
if [ ! -f "$BACKUP_PATH/metadata.json" ]; then
echo "❌ Invalid backup: metadata.json not found"
exit 1
fi
# 백업 정보 출력
echo "📋 Backup information:"
cat "$BACKUP_PATH/metadata.json" | jq '.'
}

# 2. 서비스 중지
stop_services() {
echo "⏹️ Stopping Phala services..."
kubectl scale deployment phala-node --replicas=0 -n phala-network || true
kubectl scale statefulset pruntime --replicas=0 -n phala-network || true
kubectl scale deployment pherry --replicas=0 -n phala-network || true
# 완전 종료 대기
kubectl wait --for=delete pod -l app=phala-node -n phala-network --timeo
```

```
kubectl wait --for=delete pod -l app=pruntime -n phala-network --timeout
}

# 3. 데이터 복구
restore_data() {
if [ "$RECOVERY_MODE" = "config-only" ]; then
echo "⏭️ Skipping data restore (config-only mode)"
return
fi
echo "💾 Restoring blockchain data..."
# PVC 데이터 복구
kubectl run restore-job --image=busybox --restart=Never -n phala-network \
--overrides='
{
"spec": {
"containers": [{
"name": "restore",
"image": "busybox",
"command": ["sleep", "3600"],
"volumeMounts": [{
"name": "chain-data",
"mountPath": "/data"
}]
```

```
}],
"volumes": [{
"name": "chain-data",
"persistentVolumeClaim": {
"claimName": "phala-node-pvc"
}
}]
}
}'
# 데이터 복사
kubectl wait --for=condition=ready pod restore-job -n phala-network --timeout=60
kubectl exec restore-job -n phala-network -- rm -rf /data/*
kubectl cp "$BACKUP_PATH/blockchain-data.tar.gz" phala-network/restore-job:/tmp/
kubectl exec restore-job -n phala-network -- tar -xzf /tmp/blockchain-data.tar.g
# 정리
kubectl delete pod restore-job -n phala-network
}

# 4. 설정 복구
restore_configs() {
echo "⚙️ Restoring configurations..."
# ConfigMap 업데이트
kubectl delete configmap phala-config -n phala-network || true
```

```
kubectl create configmap phala-config -n phala-network \
--from-file="$BACKUP_PATH/chain-spec.json"
# 필요시 전체 매니페스트 적용
if [ "$RECOVERY_MODE" = "full" ]; then
kubectl apply -f "$BACKUP_PATH/k8s-manifests.yaml"
fi
}

# 5. 서비스 재시작
restart_services() {
echo "🔄 Restarting Phala services..."
# 순차적 재시작
kubectl scale deployment phala-node --replicas=3 -n phala-network
kubectl wait --for=condition=available deployment phala-node -n phala-network --
kubectl scale statefulset pruntime --replicas=2 -n phala-network
kubectl wait --for=condition=ready pod -l app=pruntime -n phala-network --timeou
kubectl scale deployment pherry --replicas=1 -n phala-network
kubectl wait --for=condition=available deployment pherry -n phala-network --time
}

# 6. 상태 검증
verify_recovery() {
echo "🔍 Verifying recovery..."
```

```
# 블록체인 노드 상태 확인
local current_block=$(kubectl exec deployment/phala-node -n phala-network -- \
curl -s http://localhost:19933 \
-H "Content-Type: application/json" \
-d '{"id":1,  "jsonrpc":"2.0",  "method":"chain_getHeader"}' | \
jq -r '.result.number')
echo "📊 Current block height: $current_block"
# pRuntime 상태 확인
kubectl exec statefulset/pruntime -n phala-network -- \
curl -s http://localhost:18000/get_info | jq '.'
echo "✅ Recovery verification completed"
}


# 메인 실행
main() {
validate_environment
stop_services
restore_data
restore_configs
restart_services
verify_recovery
echo "🎉 Disaster recovery completed successfully!"
}
```

```
main "$@"
```

이 문서는 Phala Network의 배포 및 운영 가이드를 다룹니다. 마지막 문서에서는 전체 분석의 요약과 결론을 제시합니다.